



**Implementation of a Fault-Tolerant Cloud Infrastructure with
High Availability and Proactive Monitoring**

Raul Garcia-Lopez

ECE Department, California State Polytechnic University, Pomona

Abstract

This report documents the design and implementation of a fault-tolerant cloud infrastructure using virtual machines (VMs) for high availability and proactive monitoring. By leveraging tools like rsync for file synchronization, Keepalived for failover management, and Prometheus with Grafana for monitoring and visualization, the project achieves seamless service continuity. The application stack, built on React and Node.js, runs on multiple VMs to eliminate single points of failure. Additionally, the optional ELK stack enhances log analytics capabilities. This guide provides a comprehensive framework for replicating or extending the system to meet diverse requirements.

Introduction

Reliability is critical in today's digital services, where even brief outages can result in financial losses and damage to user trust. This project addresses these challenges by creating a cloud infrastructure with fault-tolerance and high availability. The system employs technologies like Keepalived, Prometheus, and Grafana to ensure real-time monitoring and automated failover, minimizing downtime. By combining modern tools for application hosting, data synchronization, and log analysis, this project provides a scalable solution adaptable to varying needs. It serves as a practical example of building robust and resilient systems for continuous service delivery.

Project Overview

- Key Features:
 - High Availability through Keepalived and VIP failover.
 - Synchronization of application files between VMs using rsync.
 - Monitoring and visualization of metrics using Prometheus and Grafana.
 - Optional: Log analytics using the ELK stack.
- Tools and Technologies:
 - Virtual Machines: VMware Workstation
 - Software: Prometheus, Grafana, Nginx, Node.js, npm, rsync, Keepalived,
(optional: Elasticsearch, Logstash, Kibana)

System Requirements

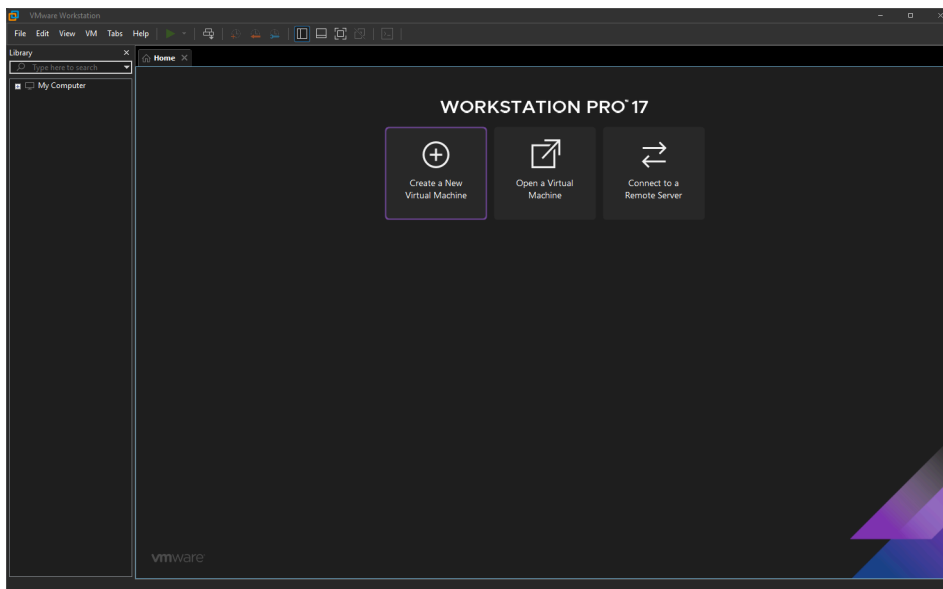
- **Processor:** Quad-core CPU (Intel i5 or AMD)
- **Memory:** 16 GB RAM
- **Storage:** 60 GB free space (20 GB per VM for three VMs)

- **Network:** Ethernet or Wi-Fi for VM communication
- **Software:** VMware Workstation Pro and GitBash

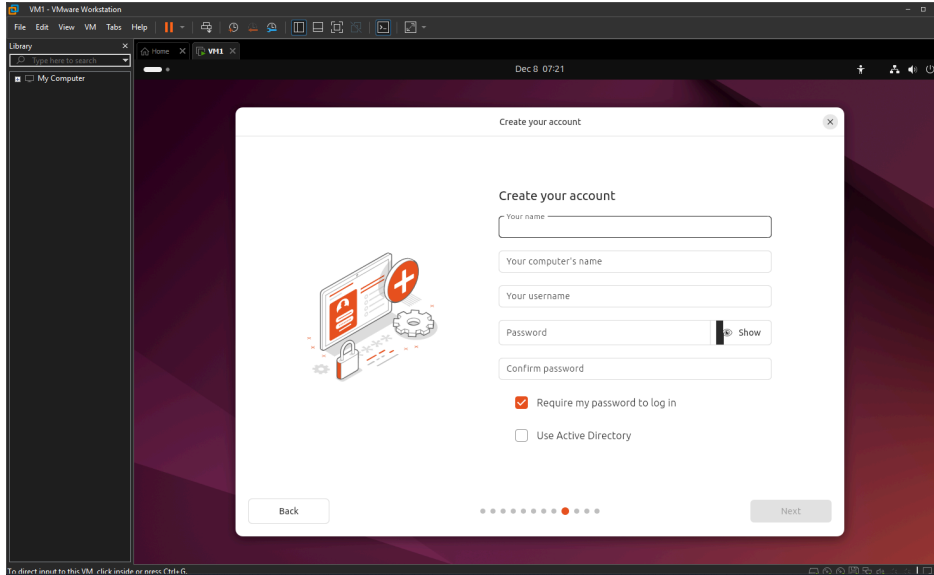
Virtual Machine Setups

Primary Virtual Machine

1. Download Ubuntu [Link Here](#) & Download/Install VMware Workstation
2. Open up VMware Workstation and Create a New Virtual Machine



3. Choose typical setup and choose Ubuntu iso image then follow setup
 - a. Name primary VM, VM1, and make sure to give VM at least 20GB
4. Now Linux VM should be created, now follow Ubuntu Installation



(Just a note to try to keep usernames and passwords the same for better flow)

5. Download & Install GitBash
6. On the VM open up the terminal and enter

```
hostname -I
```

(Take Note of IP address)

7. Update system packages and install SSH server

```
sudo apt update
sudo apt install openssh-server
sudo systemctl start ssh
sudo systemctl enable ssh
sudo systemctl status ssh
```

SSH allows you to securely log into your VMs from another system, enabling remote administration and management.

8. (Optional) On GitBash ssh into your vm you do this by entering:
 - a. ssh yourusername@yourVMipaddress, it will ask for password then confirm

```

radmin@radmin-VMware-Virtual-Platform: ~
After this operation, 1,596 kB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://us.archive.ubuntu.com/ubuntu noble-updates/main amd64 nginx-common
all 1.24. -2ubuntu7.1 [31.2 kB]
Get:2 http://us.archive.ubuntu.com/ubuntu noble-updates/main amd64 nginx amd64 1
.24.0-2ubuntu7.1 [521 kB]
Fetched 552 kB in 1s (518 kB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 151001 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.24.0-2ubuntu7.1_all.deb ...
Unpacking nginx-common (1.24.0-2ubuntu7.1) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.24.0-2ubuntu7.1_amd64.deb ...
Unpacking nginx (1.24.0-2ubuntu7.1) ...
Setting up nginx (1.24.0-2ubuntu7.1) ...
Setting up nginx-common (1.24.0-2ubuntu7.1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr
/lib/systemd/system/nginx.service.
Processing triggers for ufw (0.36.2-6) ...
Processing triggers for man-db (2.12.0-4build2) ...
radmin@radmin-VMware-Virtual-Platform:~$

```

9. Install Nginx

```

sudo apt update
sudo apt install nginx
sudo systemctl start nginx
sudo systemctl enable nginx
sudo systemctl status nginx

```

10. Configure Nginx for reverse proxy

```

sudo nano /etc/nginx/sites-available/default

```

11. Add the following configuration:

```

upstream backend {
    server 192.168.119.131:3000 fail_timeout=5s max_fails=3;
    server 192.168.119.133:3000 backup;
}

server {

```

```
listen 80;
server_name 192.168.119.140;

location / {
    proxy_pass http://backend;
}
}
```

(Save with Ctrl + O, exit with Ctrl + X)

12. Restart Nginx

```
sudo systemctl restart nginx
```

Installing **Nginx** is crucial as it acts as a reverse proxy, routing traffic through the VIP (**192.168.119.140**) to your application. The configuration ensures load balancing by directing requests to the primary server (**192.168.119.131**) and switching to the backup (**192.168.119.133**) if needed. This setup provides seamless failover, simplifies traffic management, and enhances application availability.

13. Install Keepalived

```
sudo apt update
sudo apt install keepalived
```

14. Configure Keepalived

```
sudo nano /etc/keepalived/keepalived.conf
```

15. Add the following configuration:

```
vrrp_instance VI_1 {
    state MASTER
    interface ens33
    virtual_router_id 51
    priority 100
    advert_int 1
    authentication {
        auth_type PASS
    }
}
```

```

        auth_pass sdp2024
    }
    virtual_ipaddress {
        192.168.119.140
    }
    track_script {
        chk_vrrp_state
    }
}

vrrp_script chk_vrrp_state {
    script "echo $(date) vrrp_instance VI_1 state MASTER >
/tmp/keepalived.data"
    interval 2
}

```

Installing **Keepalived** is essential for enabling high availability by managing the Virtual IP (192.168.119.140) shared between VMs. The configuration ensures that the primary server (with higher priority) maintains control of the VIP, while the backup server automatically takes over in case of failure. This setup provides seamless failover, ensuring continuous service availability and reliability.

16. Install Git, Node.js, and npm

```

sudo apt update
sudo apt install git
sudo apt install -y nodejs
sudo apt update
sudo apt install npm

```

17. Clone Portfolio Project feel free to use my [github repo](https://github.com/raulg-prog/Portfolio)

```

git clone https://github.com/raulg-prog/Portfolio

```

18. Prepare Portfolio Project

```
sudo chown -R www-data:www-data /home/raul/Portfolio
sudo chmod -R 755 /home/raul/Portfolio/.next
cd /home/raul/Portfolio
npm install
npm run build
nohup npm start &
```

Installing **Git**, **Node.js**, and **npm** is essential for setting up and hosting your Next.js application. Git is used to clone the project repository, ensuring you have the latest source code. Node.js provides the runtime environment for the application, while npm manages the dependencies needed for the project. After cloning the repository, you install dependencies with **npm install**, build the application for production with **npm run build**, and host it using **npm start**. This process allows your app to run on a production server, accessible through the Nginx reverse proxy.

19. Download and install Prometheus

```
wget
https://github.com/prometheus/prometheus/releases/download/v2.41.0/promethe
us-2.41.0.linux-amd64.tar.gz
tar -xvzf prometheus-2.41.0.linux-amd64.tar.gz
sudo mv prometheus-2.41.0.linux-amd64/prometheus /usr/local/bin/
sudo mv prometheus-2.41.0.linux-amd64/promtool /usr/local/bin/
sudo mkdir /etc/prometheus
sudo mkdir /var/lib/prometheus
sudo mv
prometheus-2.41.0.linux-amd64/{consoles,console_libraries,prometheus.yml}
/etc/prometheus/
```

20. Start Prometheus

```
sudo systemctl daemon-reload
sudo systemctl enable prometheus
sudo systemctl start prometheus
```

Prometheus is a powerful monitoring tool that collects and stores metrics from your infrastructure for real-time monitoring and alerting. The provided service configuration ensures Prometheus runs automatically and reliably. It starts only after the network is online, ensuring connectivity to monitored targets. This setup allows Prometheus to efficiently gather and store metrics, enabling detailed analysis and integration with visualization tools like Grafana.

21. Install Node Exporter

```
wget
https://github.com/prometheus/node_exporter/releases/download/v1.5.0/node_e
xporter-1.5.0.linux-amd64.tar.gz
tar -xvzf node_exporter-1.5.0.linux-amd64.tar.gz
sudo mv node_exporter-1.5.0.linux-amd64/node_exporter /usr/local/bin/
```

22. Create Node Exporter Service

```
sudo nano /etc/systemd/system/node_exporter.service
```

23. Add the following configuration:

```
[Unit]
Description=Node Exporter
After=network.target

[Service]
User=root
ExecStart=/usr/local/bin/node_exporter

[Install]
WantedBy=default.target
```

24. Start Node Exporter

```
sudo systemctl enable node_exporter
sudo systemctl start node_exporter
```

Node Exporter is a lightweight monitoring agent that collects hardware and OS-level metrics, such as CPU usage, memory consumption, and disk I/O, which are then scraped by Prometheus. The provided service configuration ensures Node Exporter runs reliably and starts automatically. It specifies that the service should launch after the network is ready, ensuring connectivity for Prometheus to scrape metrics. The **ExecStart** directive points to the Node Exporter binary, which collects metrics and exposes them on a default HTTP endpoint (**:9100**). This setup integrates seamlessly with Prometheus, providing critical system metrics for monitoring and analysis.

25. Install and Configure Filebeat

```
sudo apt update
```

```
sudo apt install filebeat -y
sudo filebeat modules enable system
sudo filebeat modules enable nginx
```

26. Configure Filebeat

```
sudo nano /etc/filebeat/filebeat.yml
```

27. Add the following configuration:

```
filebeat.config.modules:
  path: ${path.config}/modules.d/*.yaml
  reload.enabled: false

filebeat.inputs:
- type: log
  enabled: true
  paths:
    - /var/log/syslog
    - /var/log/auth.log

output.logstash:
  hosts: ["192.168.119.134:5044"]
```

28. Start Filebeat

```
sudo systemctl start filebeat
sudo systemctl enable filebeat
```

Filebeat is a lightweight log shipper that collects and forwards log files to a central processing system, such as Logstash, for further analysis. The provided configuration ensures Filebeat monitors specific log files, including system logs (`/var/log/syslog`) and authentication logs (`/var/log/auth.log`). It uses the `filebeat.inputs` section to define the paths for these logs and ensures they are actively monitored. The `output.logstash` section directs the logs to Logstash running on the monitoring server (`192.168.119.134`) at port `5044`. This setup enables centralized log management, allowing you to analyze system activity and identify issues effectively.

29. Install rsync

```
sudo apt install rsync -y
```

30. Set up Periodic Synchronization

```
sudo crontab -e
```

31. Add the following line:

```
* * * * * rsync -avz --delete /home/raul/Portfolio/  
raul@192.168.119.133:/home/raul/Portfolio/
```

Rsync is a powerful tool used for file synchronization and transfer, ensuring data consistency across servers. Installing rsync allows efficient, incremental updates between the primary server and its backup. The provided crontab entry sets up periodic synchronization, running every minute to replicate the contents of the `/home/raul/Portfolio/` directory on the primary server to the backup server (`192.168.119.133`). The `-avz` options ensure files are transferred efficiently while preserving attributes, and `--delete` removes files on the destination that no longer exist on the source, maintaining a perfect mirror. This setup ensures seamless data consistency between the servers for high availability.

Backup Virtual Machine

The setup for the backup VM is very similar to the primary VM so any changes in configurations will be shown.

1. Update system packages and install SSH server
2. SSH into VM2
3. Install Nginx & use the same configuration as the **Primary** VM
4. Install Keepalived
5. Configure Keepalived

```
sudo nano /etc/keepalived/keepalived.conf
```

6. Add the following configuration:

```
vrp_instance VI_1 {  
    state BACKUP  
    interface ens33                # Change this to your network  
interface  
    virtual_router_id 51           # Must be the same on both VMs  
    priority 90                    # Lower priority than Master  
    advert_int 1  
    authentication {
```

```

    auth_type PASS
    auth_pass sdp2024      # Must match the password in VM1
  }
  virtual_ipaddress {
    192.168.119.140        # VIP (Virtual IP Address)
  }
}

```

7. Install Git, Node.js, and npm
8. Clone Portfolio Project
9. Prepare Portfolio Project
10. Download and install Prometheus
11. Install Node Exporter
12. Create Node Exporter Service
13. Add the following configuration:

```

[Unit]
Description=Node Exporter
Wants=network-online.target
After=network-online.target

[Service]
User=node_exporter
ExecStart=/usr/local/bin/node_exporter
Restart=always

[Install]
WantedBy=default.target

```

14. Start Node Exporter
15. Install and Configure Filebeat
16. Configure Filebeat
17. Add the following configuration:

```

filebeat.config.modules:
  # Glob pattern for configuration loading
  path: ${path.config}/modules.d/*.yaml

  # Set to true to enable config reloading
  reload.enabled: false

  # Period on which to check for configuration changes
  #reload.period: 10s

filebeat.inputs:

```

```
- type: log
  enabled: true
  paths:
    - /var/log/syslog          # System logs
    - /var/log/auth.log       # Authentication logs
output.logstash:
  hosts: ["192.168.119.134:5044"] # IP address of VM3 where Logstash is
running
#output.elasticsearch:
```

18. Start Filebeat
19. Install rsync
20. Set up Periodic Synchronization
21. Add the following line:

```
* * * * * rsync -avz --delete /home/raul/Portfolio/
raul@192.168.119.133:/home/>
```

Monitor Virtual Machine

This is our final VM we are going to set up which will be used to monitor our VMs and use for failover process monitoring.

1. Update system packages and install SSH server

```
sudo apt update
sudo apt install openssh-server -y
```

2. From Git Bash, set up SSH key

```
ssh-keygen -t rsa -b 2048
ssh-copy-id raul@192.168.119.134
```

(Enter password: sdp2024)

Same as before SSH allows for remote access to the VM, with an added note that copying the key to a given VM allows for entering a VM without need of passwords.

3. Update and upgrade system packages

```
sudo apt update
sudo apt upgrade -y
```

4. Create Prometheus user and directories

```
sudo useradd --no-create-home --shell /bin/false prometheus
sudo mkdir /etc/prometheus
```

```
sudo mkdir /var/lib/prometheus
```

5. Download and Install Prometheus

```
wget  
https://github.com/prometheus/prometheus/releases/download/v2.46.0/promethe  
us-2.46.0.linux-amd64.tar.gz  
tar xvf prometheus-2.46.0.linux-amd64.tar.gz  
cd prometheus-2.46.0.linux-amd64
```

6. Move Prometheus Binaries

```
sudo mv prometheus /usr/local/bin/  
sudo mv promtool /usr/local/bin/  
sudo mv consoles /etc/prometheus  
sudo mv console_libraries /etc/prometheus  
sudo mv prometheus.yml /etc/prometheus
```

7. Set Permissions

```
sudo chown -R prometheus:prometheus /etc/prometheus  
sudo chown -R prometheus:prometheus /var/lib/prometheus
```

8. Create Prometheus Service

```
sudo nano /etc/systemd/system/prometheus.service
```

9. Add the following configuration:

```
[Unit]  
Description=Prometheus Monitoring System  
Wants=network-online.target  
After=network-online.target  
  
[Service]  
User=prometheus  
Group=prometheus  
Type=simple  
ExecStart=/usr/local/bin/prometheus --config.file  
/etc/prometheus/prometheus.yml --storage.tsdb.path=/var/lib/prometheus/  
  
[Install]  
WantedBy=multi-user.target
```

10. Configure Prometheus

```
sudo nano /etc/prometheus/prometheus.yml
```

11. Add the following configuration:

```
global:  
  scrape_interval: 15s
```

```

evaluation_interval: 15s

alerting:
  alertmanagers:
    - static_configs:
        - targets:

scrape_configs:
  - job_name: 'prometheus'
    scrape_interval: 5s
    scrape_timeout: 5s
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'node_exporter_vm1'
    static_configs:
      - targets: ['192.168.119.131:9100']

  - job_name: 'node_exporter_vm2'
    static_configs:
      - targets: ['192.168.119.133:9100']

  - job_name: 'keepalived'
    static_configs:
      - targets: ['192.168.119.140:9100']

```

12. Start Prometheus

```

sudo systemctl daemon-reload
sudo systemctl start prometheus
sudo systemctl enable prometheus

```

Installing **Prometheus** on the monitoring VM is essential for collecting and storing metrics from various targets, such as Node Exporter instances and Keepalived, enabling real-time system monitoring. The setup involves creating a dedicated Prometheus user and directories to securely manage its data, followed by installing and configuring Prometheus with a service file to ensure it starts automatically on boot. The configuration file (**prometheus.yml**) specifies global scrape intervals and defines targets, including the local Prometheus instance, Node Exporter metrics from VM1 and VM2, and Keepalived metrics via the VIP.

This setup allows Prometheus to efficiently gather system and application metrics, providing a central point for monitoring and integrating with visualization tools like Grafana.

13. Install Grafana

```
sudo apt install -y software-properties-common
wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add -
sudo add-apt-repository "deb https://packages.grafana.com/oss/deb stable
main"
sudo apt update
sudo apt install grafana -y
```

14. Start and Configure Grafana

```
sudo systemctl start grafana-server
sudo systemctl enable grafana-server
sudo grafana-cli admin reset-admin-password sdp2024
```

Installing **Grafana** on the monitoring VM is critical for visualizing the metrics collected by Prometheus and other data sources. The installation process sets up Grafana using the official package repository to ensure the latest version is installed. Once installed, Grafana is configured as a service to start automatically on boot, making it always available for monitoring. The admin password is reset for secure access, allowing you to log in and create dashboards to analyze system metrics, such as CPU usage, memory consumption, and application health, providing actionable insights and enhancing the system's observability.

15. Add Elasticsearch Repository

```
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo
apt-key add -
sudo apt-get install apt-transport-https
echo "deb https://artifacts.elastic.co/packages/7.x/apt stable main" | sudo
tee -a /etc/apt/sources.list.d/elastic-7.x.list
sudo apt-get update
sudo apt-get install elasticsearch
```

16. Configure Elasticsearch

```
sudo nano /etc/elasticsearch/elasticsearch.yml
```

17. Add the following configuration:

```
cluster.name: fault-tolerance
node.name: elk-node-1
path.data: /var/lib/elasticsearch
path.logs: /var/log/elasticsearch
network.host: 0.0.0.0
http.port: 9200
discovery.seed_hosts: ["192.168.119.134"]
cluster.initial_master_nodes: ["elk-node-1"]
```

18. Optimize and Start Elasticsearch

```
sudo sysctl -w vm.max_map_count=262144
echo "vm.max_map_count=262144" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
sudo systemctl daemon-reload
sudo systemctl start elasticsearch
sudo systemctl enable elasticsearch
```

Installing **Elasticsearch** as part of the ELK stack on the monitoring VM is essential for indexing, storing, and searching log data collected from various sources. The setup involves adding the official Elasticsearch repository to ensure access to the latest version, configuring the service with key parameters such as cluster and node names, data and log storage paths, and network settings for communication. Optimizing memory settings enhances performance for handling large volumes of log data. By enabling Elasticsearch as a service, it starts automatically on boot, forming the foundation of the ELK stack for centralized log management, enabling efficient querying and integration with tools like Logstash and Kibana for detailed log analysis.

19. Install Kibana

```
sudo apt-get install kibana
```

20. Configure Kibana

```
sudo nano /etc/kibana/kibana.yml
```

21. Add the following configuration:

```
server.port: 5601
server.host: "0.0.0.0"
server.name: "kibana-server"
elasticsearch.hosts: ["http://192.168.119.134:9200"]
kibana.index: ".kibana"
```

22. Start Kibana

```
sudo systemctl start kibana
sudo systemctl enable kibana
```

Installing **Kibana** as part of the ELK stack on the monitoring VM provides a user-friendly interface for visualizing and analyzing log data stored in Elasticsearch. The setup involves configuring Kibana to connect to the Elasticsearch instance on the monitoring VM, specifying key parameters like the server port, host, and Elasticsearch endpoint. By enabling Kibana as a service, it starts automatically on boot, ensuring accessibility for creating dashboards, visualizations, and queries. This integration with Elasticsearch completes the ELK stack, enabling centralized log analysis and offering insights into system performance and application logs.

23. Install Logstash

```
sudo apt-get install logstash
```

24. Configure Logstash Input

```
sudo nano /etc/logstash/conf.d/02-beats-input.conf
```

25. Add the following configuration:

```
input {
  beats {
    port => 5044
  }
}
```

26. Configure Logstash Output

```
sudo nano /etc/logstash/conf.d/10-elasticsearch-output.conf
```

27. Add the following configuration:

```
output {
  elasticsearch {
    hosts => ["192.168.119.134:9200"]
    user => "elastic"
    password => "your_elasticsearch_password"
    manage_template => false
    index => "%{[@metadata][beat]}-%{+YYYY.MM.dd}"
  }
  stdout { codec => rubydebug }
}
```

28. Start Logstash

```
sudo systemctl enable logstash
sudo systemctl start logstash
```

Installing **Logstash** as part of the ELK stack on the monitoring VM enables centralized log processing by receiving, filtering, and forwarding log data to Elasticsearch. The configuration involves setting up an input pipeline to receive logs from Filebeat on port 5044 and an output pipeline to send processed data to Elasticsearch. The Elasticsearch output configuration includes essential parameters like the Elasticsearch host, user authentication, and indexing rules. By enabling Logstash as a service, it automatically starts on boot, acting as the intermediary between log shippers like Filebeat and Elasticsearch, ensuring structured and efficient log ingestion for analysis.

Monitor Visualization

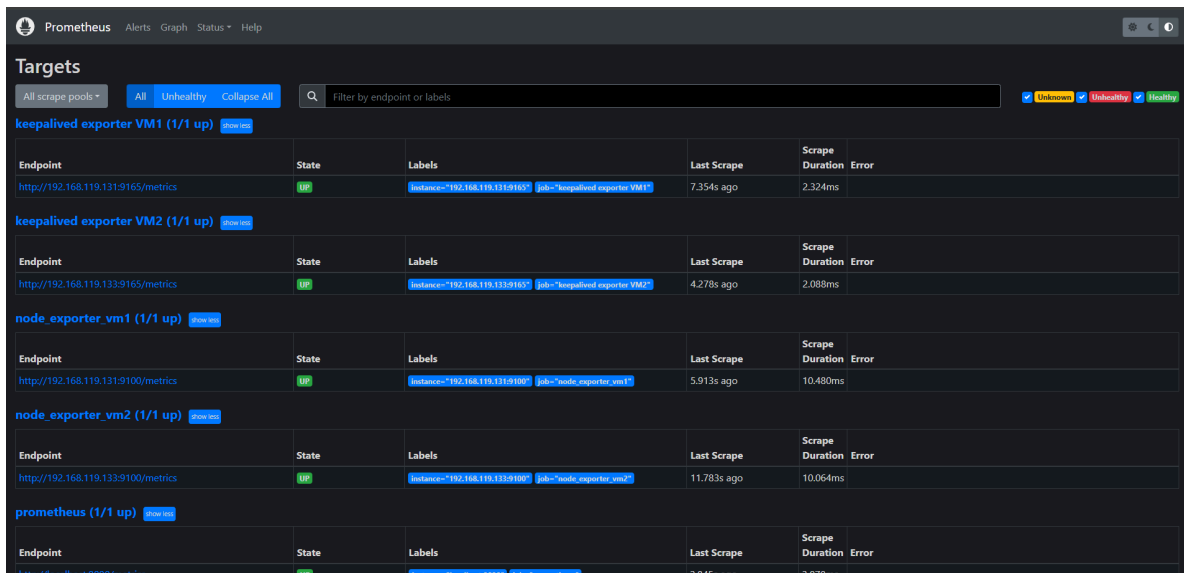
A couple of things to set clear, before we can monitor our failovers we must be hosting the website through npm. We do this on VM1 by:

```
cd /home/raul/Portfolio
npm run build
nohup npm start &
```

Once we have all 3 VMs up we can begin monitoring

Prometheus

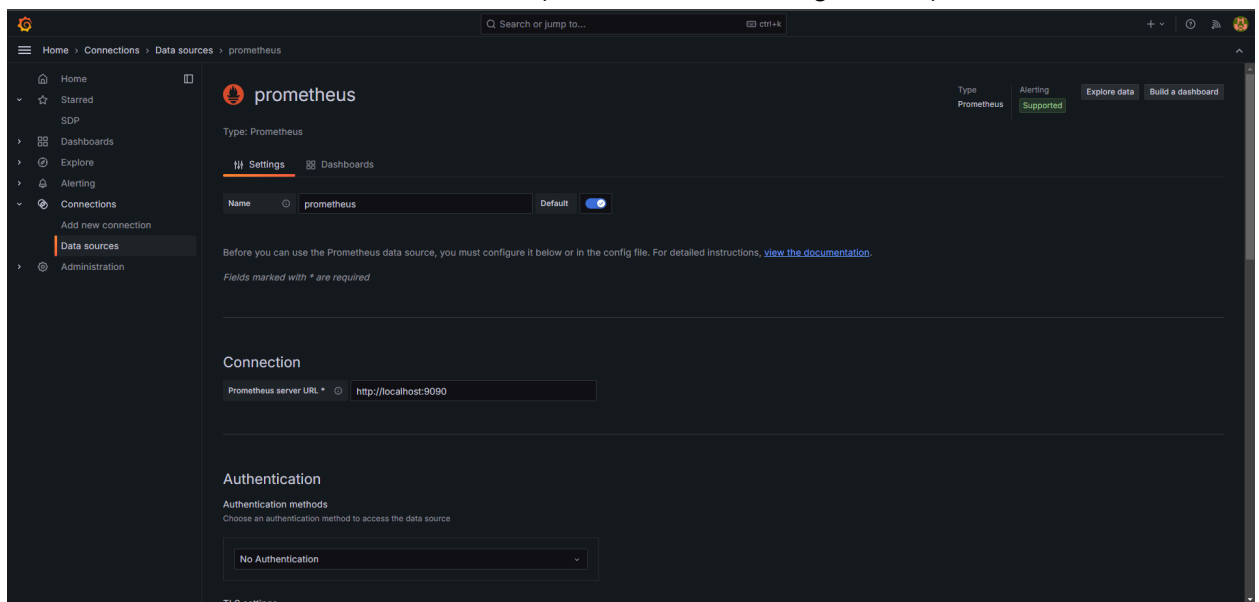
Let's start by verifying our targets are being monitored on prometheus on my cluster I visit <http://192.168.119.134:9090/targets?search=> this would be VM3'sipaddr:9090/targets



The screenshot shows the Prometheus web interface at the 'Targets' page. It lists several scrape pools, each with a table of endpoints. The endpoints are all in a 'UP' state. The labels for each endpoint include the instance name and the target IP address.

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
keepalived exporter VM1 (1/1 up)					
http://192.168.119.133:9165/metrics	UP	instance="192.168.119.133:9165" job="keepalived exporter VM1"	7.354s ago	2.324ms	
keepalived exporter VM2 (1/1 up)					
http://192.168.119.133:9165/metrics	UP	instance="192.168.119.133:9165" job="keepalived exporter VM2"	4.278s ago	2.088ms	
node_exporter_vm1 (1/1 up)					
http://192.168.119.133:9100/metrics	UP	instance="192.168.119.133:9100" job="node_exporter_vm1"	5.913s ago	10.480ms	
node_exporter_vm2 (1/1 up)					
http://192.168.119.133:9100/metrics	UP	instance="192.168.119.133:9100" job="node_exporter_vm2"	11.783s ago	10.064ms	
prometheus (1/1 up)					
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus"	2.845s ago	3.070ms	

- Once logged in you will see **Data Sources** on the left hand side, go there
- Choose Prometheus as the Data Source (it should like the image below)



The screenshot shows the Prometheus configuration page in the Grafana UI. The left sidebar shows the 'Data sources' menu item. The main content area shows the configuration for the 'prometheus' data source.

prometheus

Type: Prometheus

Settings

Name: prometheus Default: ☒

Before you can use the Prometheus data source, you must configure it below or in the config file. For detailed instructions, [view the documentation](#).

Fields marked with * are required

Connection

Prometheus server URL: http://localhost:9090

Authentication

Authentication methods

Choose an authentication method to access the data source

No Authentication

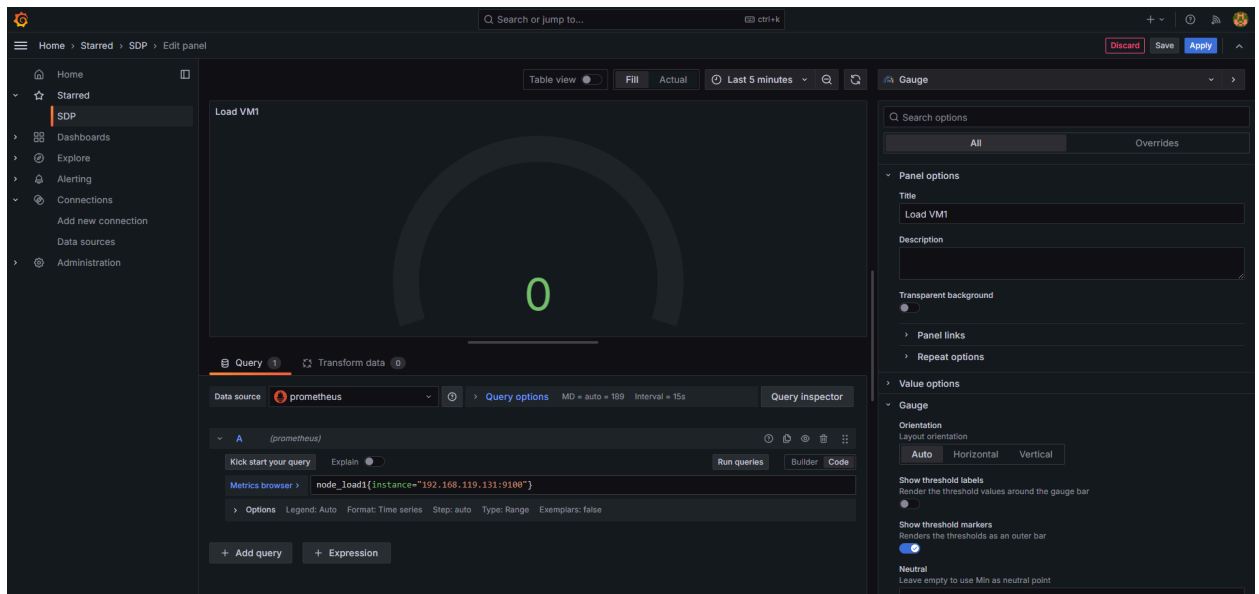
TLS settings

- Now that Prometheus is a Data Source for Grafana we can begin pulling metrics from Prometheus to start Visualizing the Data
- Go to the main menu of Grafana and on the left hand side click **Dashboards**
- Now make a new Dashboard and Create a Visualization
- Choose Prometheus as your data source
- Our first query will be Node Load =

```
node_load1{instance="192.168.119.131:9100"}
```

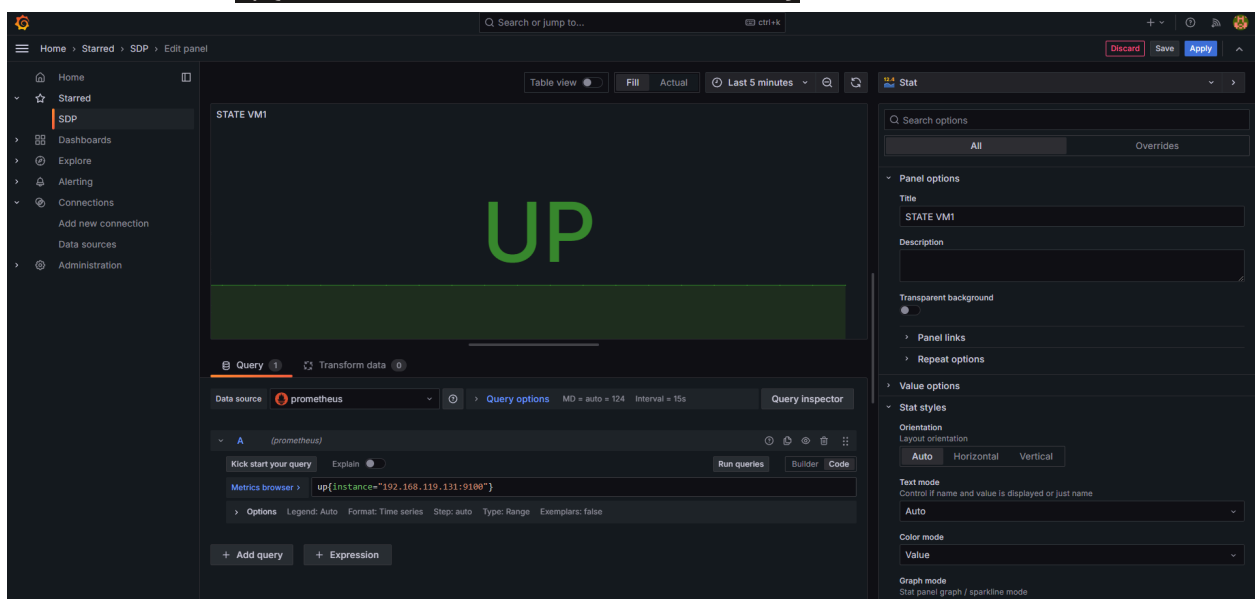
This query represents the system's load average over the past 1 minute. It measures the average number of processes that are either running or waiting for CPU time.

- I chose to Visualize it as a gauge which can be chosen on the right hand side



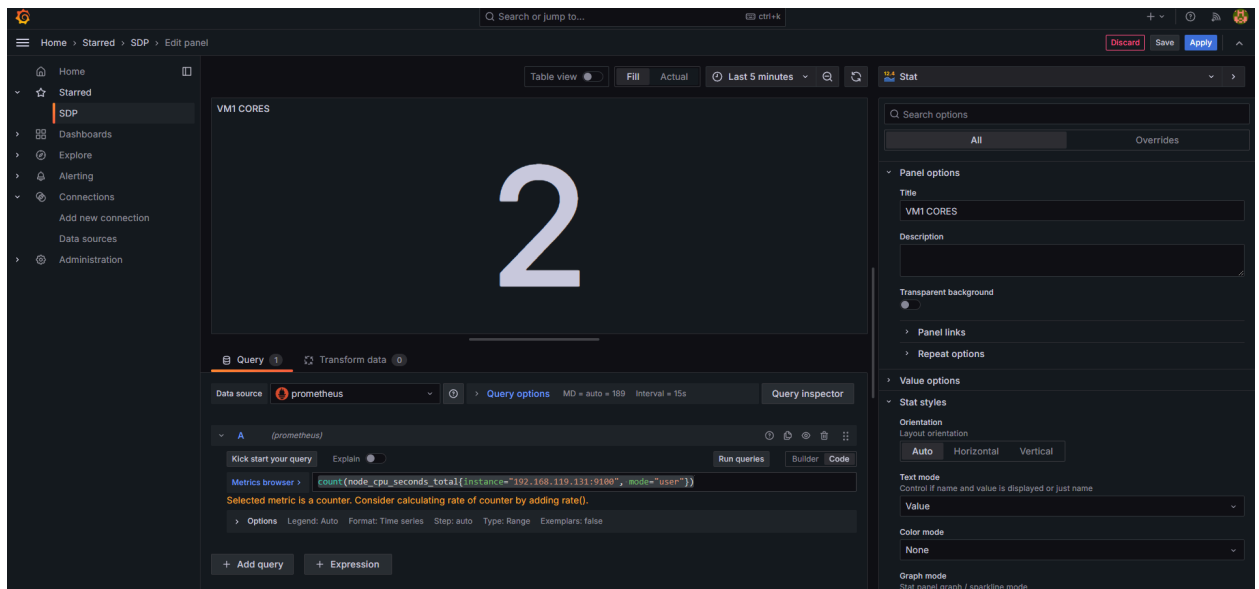
- State of a node =

```
up{instance="192.168.119.131:9100"}
```



This query is used to ensure that Prometheus is successfully scraping metrics from the specified Node Exporter instance.

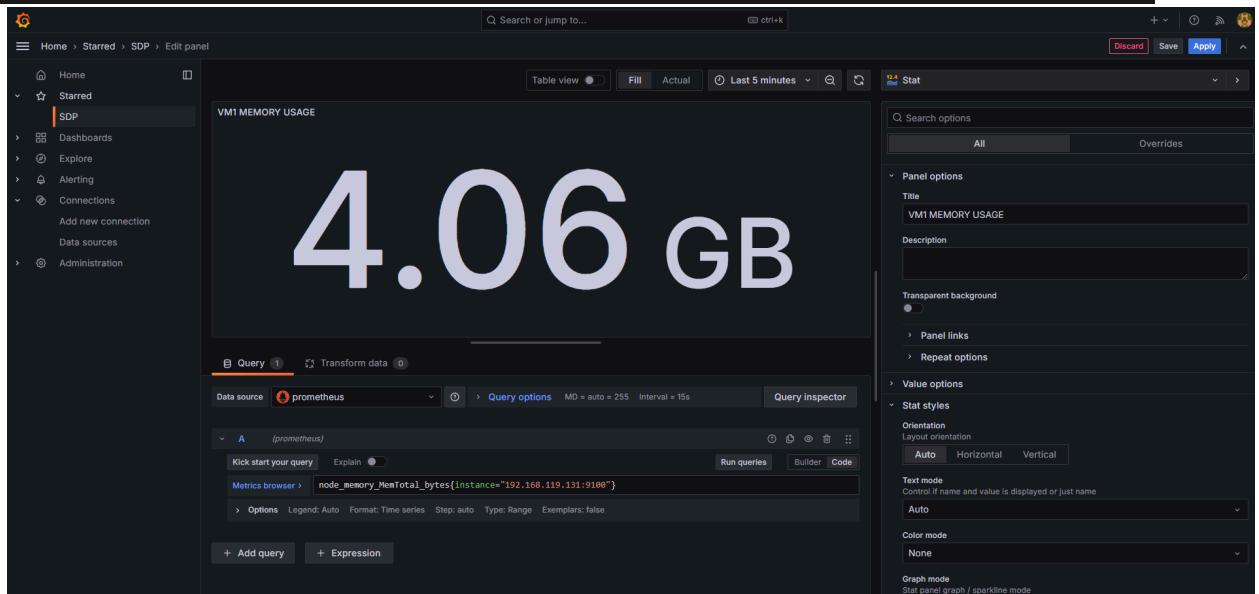
- CPU cores = `count(node_cpu_seconds_total{instance="192.168.119.131:9100", mode="user"})`



This query helps determine how many CPU cores are available on the target machine for user-level workloads.

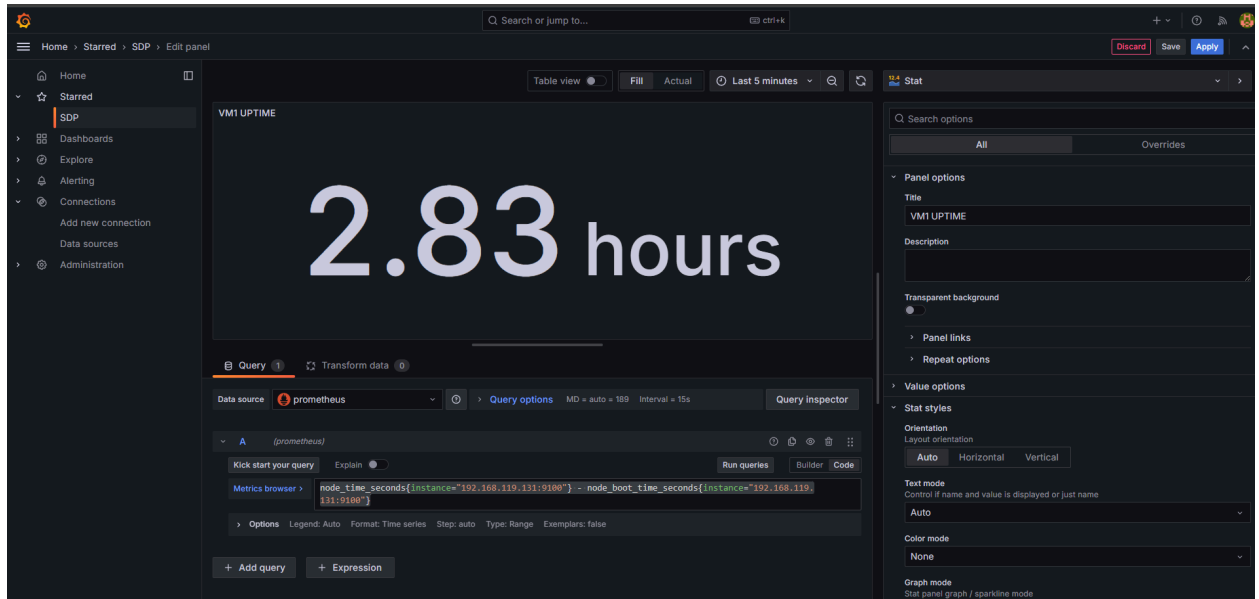
- Memory Usage =

`node_memory_MemTotal_bytes{instance="192.168.119.131:9100"}`



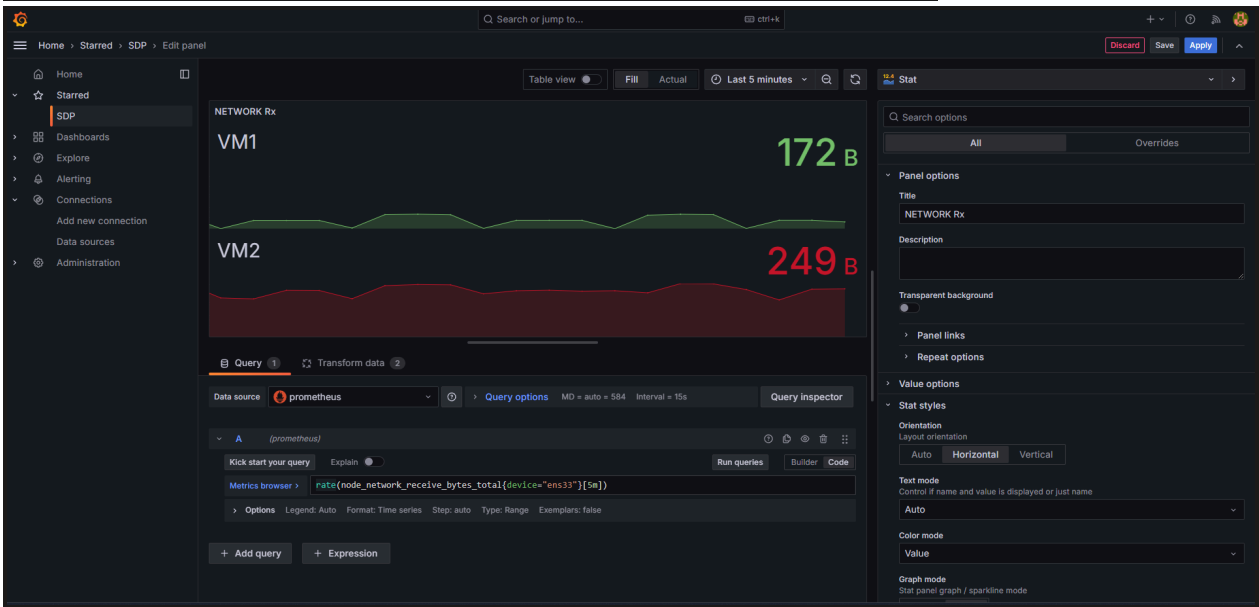
This query is used to monitor the amount of physical memory on a server, which is essential for understanding resource capacity.

- VM uptime = `node_time_seconds{instance="192.168.119.131:9100"} - node_boot_time_seconds{instance="192.168.119.131:9100"}`



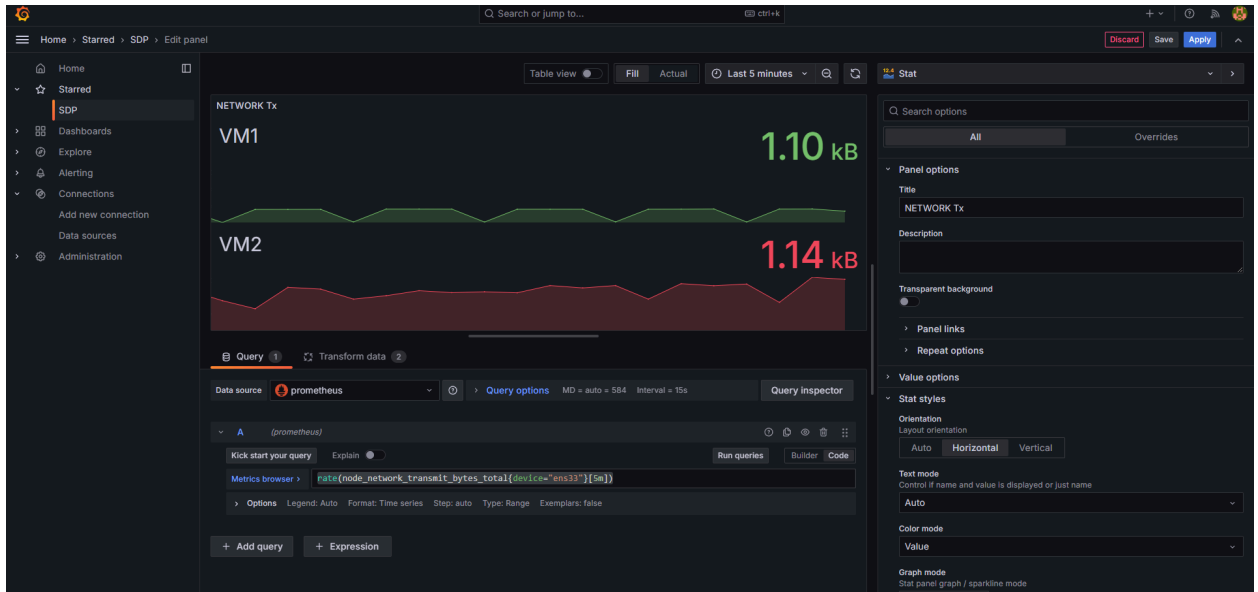
This query provides the exact duration the system has been running without a reboot.

- Network Reception = `rate(node_network_receive_bytes_total{device="ens33"}[5m])`



This query helps track how much data is being received by the interface, providing real-time insights into network activity.

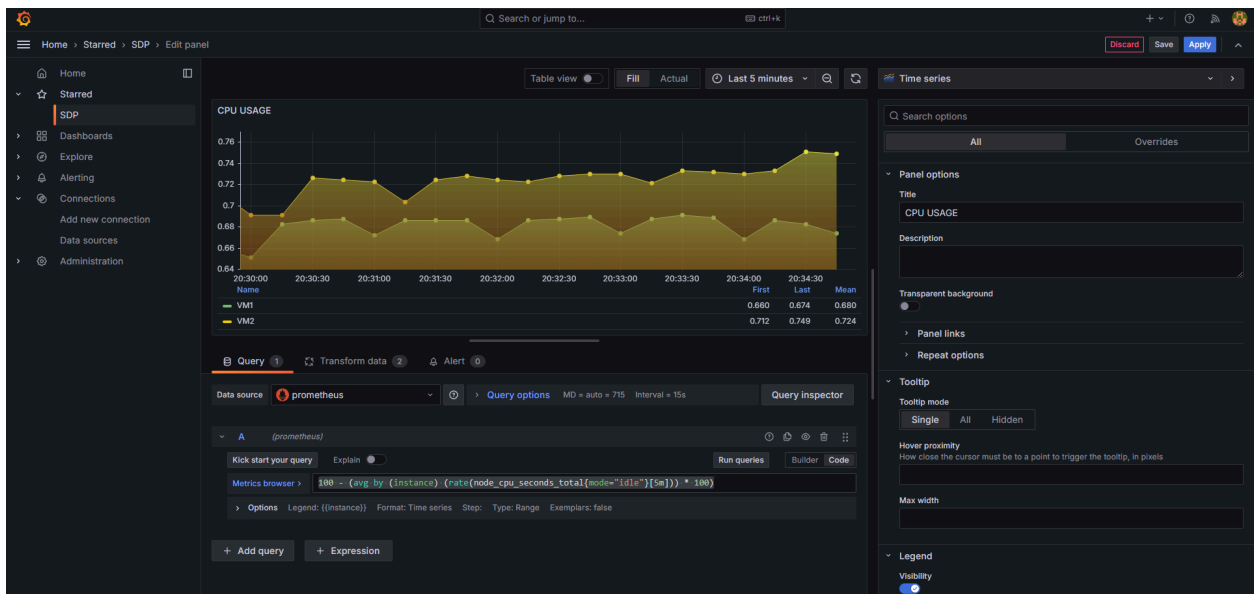
- Network Transmission = `rate(node_network_transmit_bytes_total{device="ens33"}[5m])`



This query helps track how much data is being sent by the network interface, providing real-time insights into outgoing network usage.

- CPU Usage = $100 - (\text{avg by (instance)})$

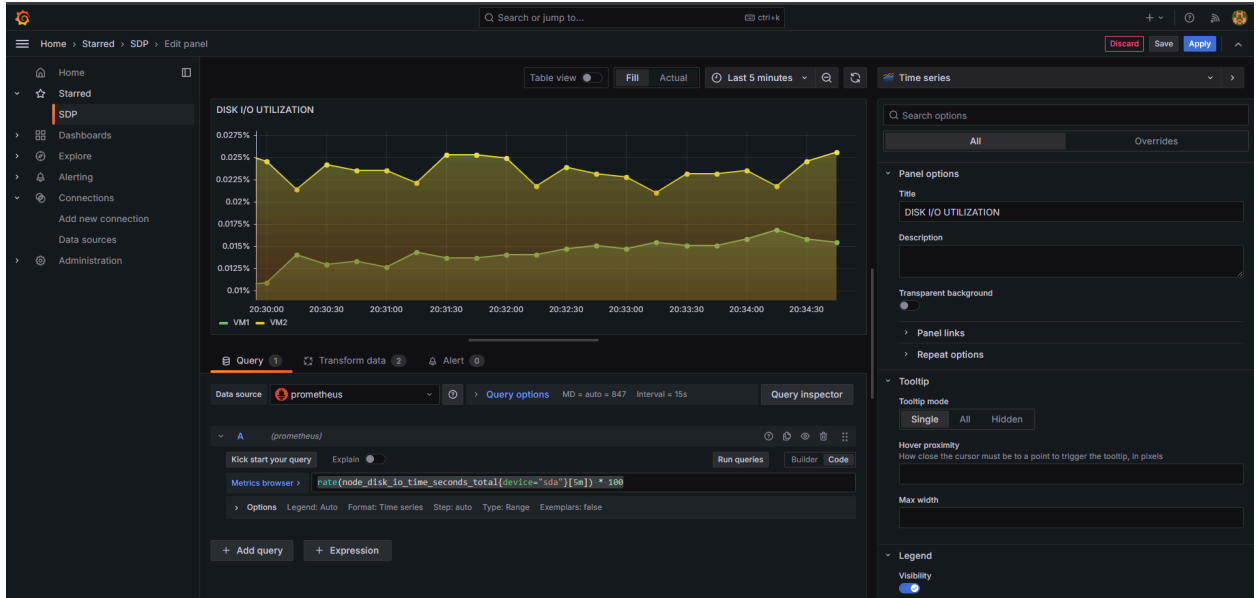
$(\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{"idle"}\}[5\text{m}])) * 100)$



This query helps track how actively the CPU is being used, providing insights into system performance and workload distribution.

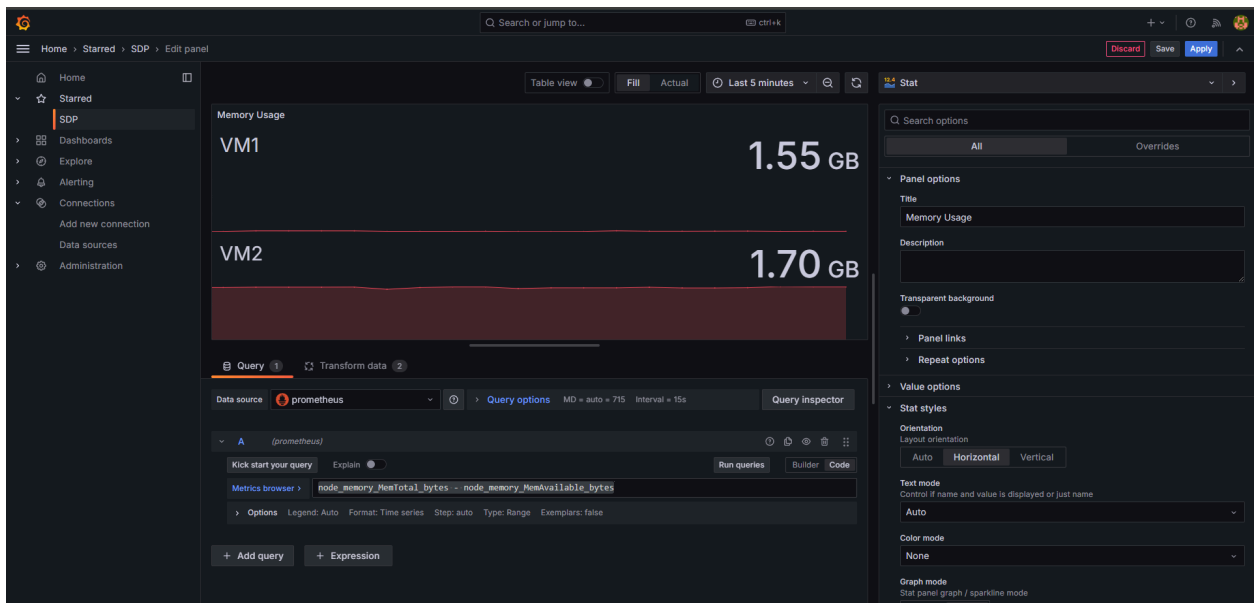
- Disk I/O Utilization =

$\text{rate}(\text{node_disk_io_time_seconds_total}\{\text{device}=\text{"sda"}\}[5\text{m}]) * 100$



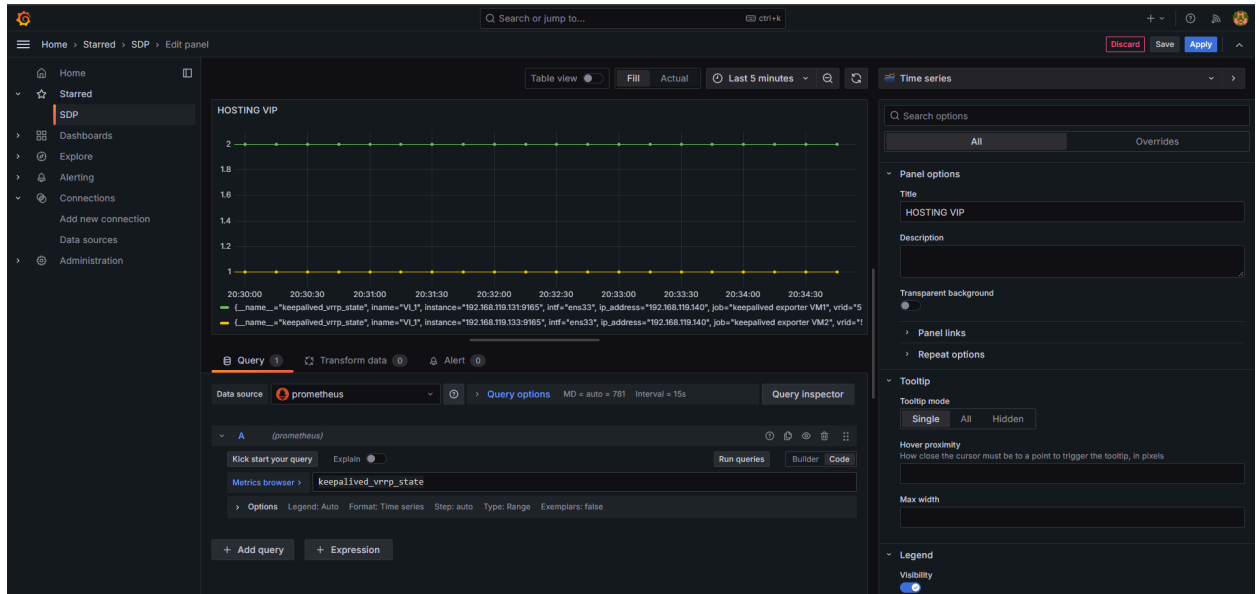
This query indicates how heavily a disk is being used for read/write operations.

- Memory Usage = `node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes`



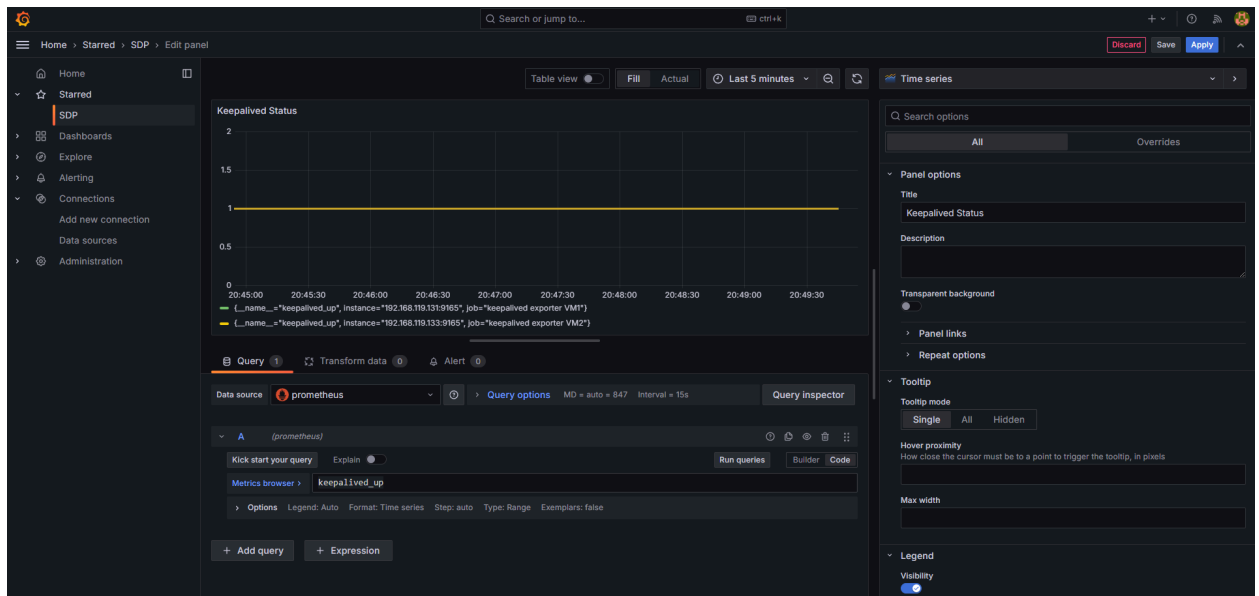
This query provides a direct measure of how much memory is actively being used on the system.

- VIP Host = `keepalived_vrrp_state`



This query ensures that the MASTER and BACKUP roles are correctly assigned and functioning as intended.

- KeepAlived Status = `keepalived_up`



This query ensures that Keepalived is active and functioning correctly on all configured nodes.

Monitor Alerting

We have created Monitoring visually now we will move onto creating alerts for our failover process.

1. SSH into VM3 our Monitor VM
2. Download and Extract Alertmanager

```
wget
https://github.com/prometheus/alertmanager/releases/download/v0.25.0/alertmanager-0.25.0.linux-amd64.tar.gz
```

3. Extract the contents:

```
tar -xvzf alertmanager-0.25.0.linux-amd64.tar.gz
cd alertmanager-0.25.0.linux-amd64
```

4. Move the main Alertmanager binaries to the system's PATH for easier access:

```
sudo mv alertmanager /usr/local/bin/
sudo mv amtool /usr/local/bin/
```

Important Note: Ensure you know the location of the configuration file (`alertmanager.yml`) after this step. It is located in the extracted directory (`alertmanager-0.25.0.linux-amd64`) unless moved.

5. Open the AlertManager configuration file:

```
nano /home/raul/alertmanager-0.25.0.linux-amd64/alertmanager.yml
```

6. Use the following configuration as an example to route alerts to the webhook listener on VM2:

```
route:
  group_by: ['alertname']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 1h
  receiver: 'webhook_receiver'

receivers:
- name: 'webhook_receiver'
  webhook_configs:
    - url: 'http://192.168.119.133:5001/' # Webhook endpoint on VM2
```

7. Add Alertmanager details to Prometheus:

```
sudo nano /etc/prometheus/prometheus.yml
```

8. Include the following under the **alerting** section:

```
alerting:
  alertmanagers:
    - static_configs:
      - targets: ['localhost:9093']
```

9. Load alert rules:

```
rule_files:
  - "/etc/prometheus/alert.rules"
```

10. Open the alert rules file:

```
sudo nano /etc/prometheus/alert.rules
```

11. Add an alert for when VM1 goes down:

```
groups:
  - name: instance_down_alerts
    rules:
      - alert: InstanceDown
        expr: up{instance="192.168.119.131:9100"} == 0
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "Instance VM1 is down"
          description: "VM1 (192.168.119.131) has been unreachable for 1
minute."
```

12. Create the service file:

```
sudo nano /etc/systemd/system/alertmanager.service
```

13. Add the following configuration:

```
[Unit]
Description=Alertmanager
Wants=network-online.target
After=network-online.target

[Service]
ExecStart=/usr/local/bin/alertmanager \
--config.file=/home/raul/alertmanager-0.25.0.linux-amd64/alertmanager.yml
Restart=always

[Install]
WantedBy=multi-user.target
```

14. Reload systemd:

```
sudo systemctl daemon-reload
sudo systemctl enable alertmanager
```

15. Start the service:

```
sudo systemctl start alertmanager
```

16. Reload the Prometheus service

```
sudo systemctl reload prometheus
```

17. Restart Prometheus (if necessary):

```
sudo systemctl restart prometheus
```

18. Verify Prometheus is running:

```
sudo systemctl status prometheus
```

19. Exit VM3 and SSH into VM2

20. Install the required Flask library for Python:

```
sudo apt install python3-flask -y
```

21. Open a new file:

```
nano webhook_listener.py
```

22. Add the following code:

```
import os
from flask import Flask, request

app = Flask(__name__)

@app.route('/', methods=['POST'])
def alert():
    data = request.json
    if data:
        # Log incoming alerts for debugging
        with open("/home/raul/webhook.log", "a") as log_file:
            log_file.write(str(data) + "\n")

        # Change directory and execute commands
        os.chdir("/home/raul/Portfolio")
        os.system("npm run build && nohup npm start &")
        return "Alert received and commands executed", 200
    return "No data received", 400

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5001)
```

23. Set sudo permissions

```
sudo visudo
```

24. Add

```
# Allow members of group sudo to execute any command
raul ALL=(ALL) NOPASSWD: /usr/bin/npm, /bin/cd
```

25. Open the service file:

```
sudo nano /etc/systemd/system/webhook_listener.service
```

26. Add the following configuration:

```
[Unit]
Description=Webhook Listener Service
After=network.target

[Service]
User=raul
ExecStart=/usr/bin/python3 /home/raul/webhook_listener.py
Restart=always

[Install]
WantedBy=multi-user.target
```

27. Run the following commands to enable and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable webhook_listener.service
sudo systemctl start webhook_listener.service
```

28. Check the status of the service to ensure it's running:

```
sudo systemctl status webhook_listener.service
```

29. Some troubleshooting: If the service fails to start due to a port conflict, use the following commands:

30. Find the process using port 5001:

```
sudo lsof -i :5001
```

31. Kill the conflicting process:

```
sudo kill -9 <PID>
```

32. Restart the webhook listener:

```
sudo systemctl start webhook_listener.service
```

33. Exit VM2 and SSH into VM3

34. Open the alert rules file:

```
sudo nano /etc/prometheus/alert.rules
```

35. Add a new alert for VM1's high CPU usage:

```
groups:
  - name: vm1_high_cpu_alert
    rules:
      - alert: HighCPUUsageVM1
        expr: 100 - (avg by (instance)
(rate(node_cpu_seconds_total{mode="idle",
instance="192.168.119.131:9100"}[5m])) * 100) > 95
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "High CPU Usage on VM1"
          description: "CPU usage on VM1 (192.168.119.131) has exceeded 95%
for 1 minute."
```

36. Open the Alertmanager configuration file:

```
sudo nano /home/raul/alertmanager-0.25.0.linux-amd64/alertmanager.yml
```

37. Add a new route for the high CPU alert:

```
route:
  group_by: ['alertname']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 1h
  receiver: 'shutdown_vm1'

receivers:
  - name: 'shutdown_vm1'
    webhook_configs:
      - url: 'http://192.168.119.131:5001/' # Webhook endpoint on VM1
```

38. Restart Alertmanager:

```
sudo systemctl restart alertmanager
```

39. Now exit VM3 and SSH into VM1

40. Install the required Flask library for Python:

```
sudo apt install python3-flask -y
```

41. Open a new file:

```
sudo nano /home/raul/shutdown_webhook.py
```

42. Add the following code:

```
import os
from flask import Flask, request

app = Flask(__name__)

@app.route('/', methods=['POST'])
def shutdown():
    data = request.json
    if data:
        # Log incoming alerts for debugging
        with open("/home/raul/webhook.log", "a") as log_file:
            log_file.write(str(data) + "\n")

        # Shutdown the system
        os.system("sudo shutdown now")
        return "Shutdown command executed", 200
    return "No data received", 400

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5001)
```

43. Change Sudo permissions

```
sudo visudo
```

44. Add

```
raul ALL=(ALL) NOPASSWD: /sbin/shutdown
```

45. Configure and enable the service:

```
sudo nano /etc/systemd/system/shutdown_webhook.service
```

```
[Unit]
Description=Shutdown Webhook Listener
After=network.target

[Service]
User=raul
ExecStart=/usr/bin/python3 /home/raul/shutdown_webhook.py
Restart=always

[Install]
WantedBy=multi-user.target
```

46. Reload systemd and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable shutdown_webhook.service
sudo systemctl start shutdown_webhook.service
```

Failover Process Overview

The failover process ensures reliable monitoring of virtual machine health. Nginx serves as a reverse proxy to manage traffic between VM1 and VM2. Keepalived enables high availability by automatically transferring the shared Virtual IP (VIP) to VM2 if VM1 becomes unavailable, minimizing service disruptions.

Alerts play a key role in this setup: the first triggers when VM1 goes offline, ensuring that VM2 seamlessly takes over hosting the application. The second alert monitors CPU usage on VM1, shutting it down if usage exceeds 95% for one minute and activating the failover mechanism. This system showcases resilience by maintaining uninterrupted service during failures.

Conclusion

This project built a fault-tolerant cloud infrastructure that ensures high availability and seamless failover with tools like Keepalived, rsync, Prometheus, and Grafana. It achieved key goals like creating a reliable failover mechanism, synchronizing data between VMs, and providing clear metrics for real-time monitoring. The optional ELK stack added value with enhanced log analysis. In the future, scaling automation and predictive analytics could improve performance and further reduce downtime, making the system even more adaptable to real-world needs..

Sources

1. **Prometheus Documentation**
The Prometheus Authors. (n.d.). *Prometheus Documentation*. Retrieved from <https://prometheus.io/docs/introduction/overview/>
2. **Grafana Documentation**
Grafana Labs. (n.d.). *Grafana Documentation*. Retrieved from <https://grafana.com/docs>
3. **Flask Framework Documentation**
Ronacher, A., & The Pallets Projects. (n.d.). *Flask Documentation*. Retrieved from <https://flask.palletsprojects.com>
4. **Nginx Documentation**
Nginx, Inc. (n.d.). *Nginx Documentation*. Retrieved from <https://nginx.org/en/docs>
5. **Keepalived Documentation**
Keepalived Authors. (n.d.). *Keepalived Documentation*. Retrieved from <http://www.keepalived.org/documentation.html>
6. **Alertmanager Documentation**
The Prometheus Authors. (n.d.). *Alertmanager Documentation*. Retrieved from <https://prometheus.io/docs/alerting/latest/alertmanager/>
7. **Stress Command Documentation**
Torrance, C. (n.d.). *Stress Command Documentation*. Retrieved from <https://man7.org/linux/man-pages/man1/stress.1.html>
8. **Node.js Documentation**
OpenJS Foundation. (n.d.). *Node.js Documentation*. Retrieved from <https://nodejs.org/en/docs>
9. **Filebeat Documentation**
Elastic. (n.d.). *Filebeat Documentation*. Retrieved from <https://www.elastic.co/guide/en/beats/filebeat/current/index.html>
10. **High Availability for the LAMP Stack**
Chisnall, J. (2010). *High Availability for the LAMP Stack: Eliminate Single Points of Failure and Increase Uptime for Your Linux, Apache, MySQL, and PHP Based Web Applications*. Packt Publishing.
11. **Designing Data-Intensive Applications**
Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.